

Approximate Nearest-Neighbor Search on the Tenstorrent Wormhole Architecture

Master's Degree in Computer Science

Barbalata Radu Ionut (2002688)

Academic Year 2025/2026



SAPIENZA
UNIVERSITÀ DI ROMA



Table of Contents

1 Introduction

► Introduction

► Background

► Design and Implementation

► Methodology

► Results

► Conclusions



Motivation

1 Introduction

- Vector search retrieves the items most similar to a query embedding: it is the retrieval step of RAG, recommendation, and semantic search
- Exact search compares the query with every vector: too slow for large collections
- Approximate nearest-neighbor (ANN) indexes trade a little recall for much higher speed
- Today ANN search runs on CPUs and GPUs
- **Tenstorrent** builds tile-based AI accelerators: can they also serve vector search, and at what performance and energy cost?



Research Questions

1 Introduction

1. Which ANN index structure suits the Tenstorrent architecture and programming model?
2. Can an IVF-based pipeline be implemented efficiently with tiles, explicit data movement, and distributed Tensix cores?
3. What Recall@10 and throughput does it reach compared with multicore CPUs and an NVIDIA A100?
4. How does its energy compare at matched recall?



Contributions

1 Introduction

- Analysis of the main ANN index families for Tenstorrent $\Rightarrow$ IVF-Flat
- First ANN implementation on Tenstorrent, on a Wormhole n150
- Comparison with Faiss on 56/112 CPU cores and an A100 at matched Recall@10
- Throughput, workload energy, and a stage-level analysis of the bottlenecks



Table of Contents

2 Background

- ▶ Introduction
- ▶ **Background**
- ▶ Design and Implementation
- ▶ Methodology
- ▶ Results
- ▶ Conclusions



Approximate Nearest-Neighbor Search

2 Background

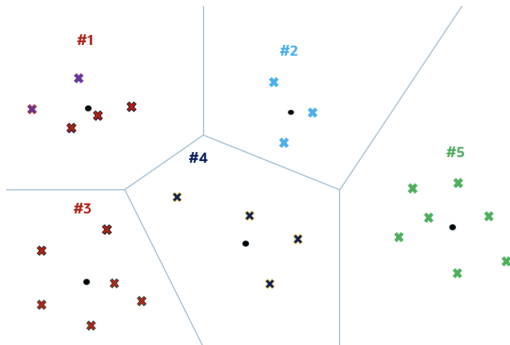
- Database of n vectors in $\mathbb{R}^d$, query q : find the k most similar vectors
- Similarity: inner product on normalized vectors (cosine)
- Quality measured by **Recall@10**: fraction of the true 10 nearest neighbors that are returned
- Main index families: inverted files (IVF), graphs (HNSW, CAGRA), quantization (IVF-PQ, ScaNN)



Inverted File Index (IVF)

2 Background

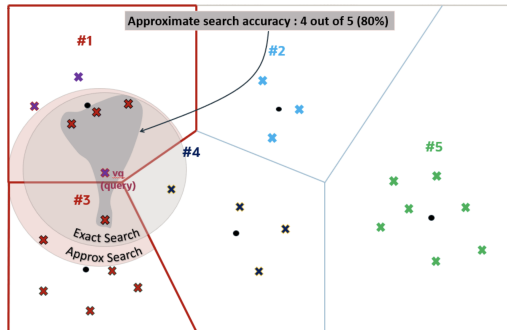
- k -means splits the database into N_{list} cells
- **Coarse search:** compare the query with all centroids
- **Fine search:** scan only the N_{probe} closest lists
- Larger N_{probe} : higher recall, more work





Why Approximate: Missed Neighbors

2 Background



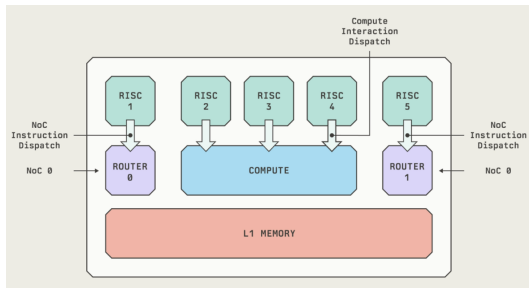
A true neighbor in an unprobed cell is missed: N_{probe} trades recall for speed



Tenstorrent Wormhole n150

2 Background

- Grid of Tensix cores connected by a network-on-chip (NoC)
- Each core: five RISC-V cores, local L1 SRAM, matrix engine on 32×32 tiles
- 12 GB GDDR6 device DRAM, attached via PCIe
- TT-Metalium: reader, compute, and writer kernels linked by circular buffers





Why IVF-Flat?

2 Background

- **Graphs** (HNSW, CAGRA): irregular, data-dependent traversal, poor fit for tiles
- **Quantization** (IVF-PQ, ScaNN): codebook lookups map poorly onto the matrix engine
- **IVF-Flat**: dense products between query tiles and contiguous lists
 - regular, fixed-shape computation
 - predictable, streamable memory access
 - the irregular part (list selection) can be handled on the host



Table of Contents

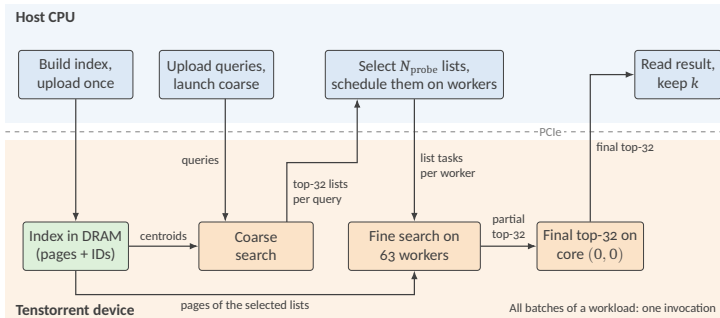
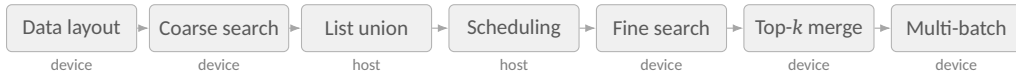
3 Design and Implementation

- ▶ Introduction
- ▶ Background
- ▶ **Design and Implementation**
- ▶ Methodology
- ▶ Results
- ▶ Conclusions



Overview

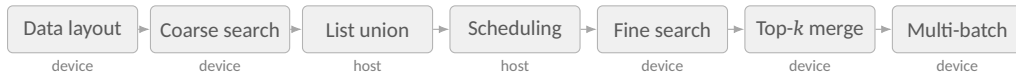
3 Design and Implementation





Running Example

3 Design and Implementation



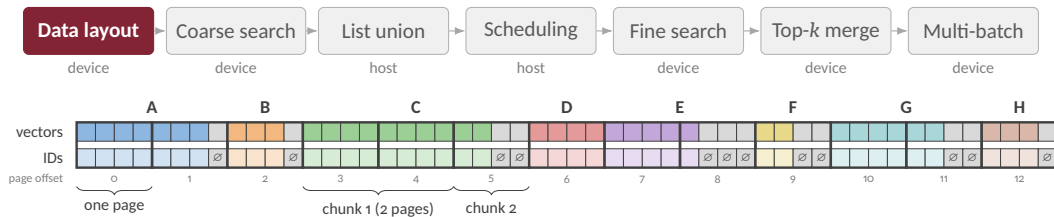
	Example	Implementation
Tile size	4×4	32×32
Queries per batch	$4 (q_0 - q_3)$	32
Inverted lists	8 (A-H)	512-2048
Lists per query N_{probe}	2	1-32
Top- k state	top-2	top-32
Fine-search workers	3 (W1-W3)	63
Chunk limit	2 pages	128 pages

Selections: $q_0\{A, C\}$, $q_1\{A, B\}$, $q_2\{C, E\}$, $q_3\{A, D\}$



Step 1: Tiled Data Layout

3 Design and Implementation

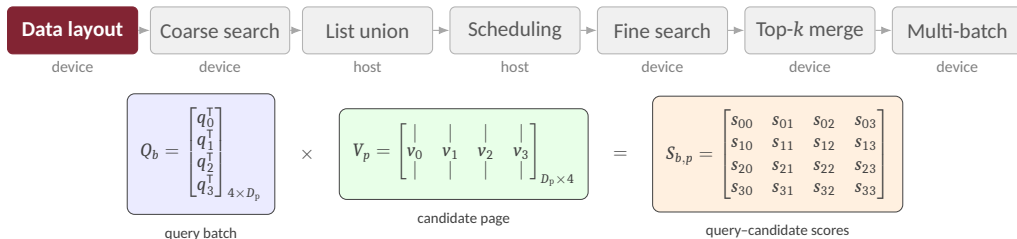


- Lists stored as **pages** of 32 vectors, dimensions padded to multiples of 32 (100 → 128)
- Each page has an identifier tile; padded slots carry a sentinel and are masked to -10000
- Index uploaded once to device DRAM; packed buffers addressed by page offset



Step 1: Query Batch $\times$ Page

3 Design and Implementation

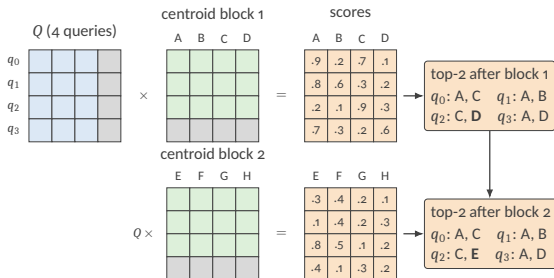
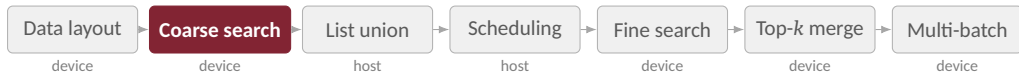


- A batch of 32 queries is the row operand, a page of 32 vectors the column operand
- One tile product scores every query of the batch against every vector of the page



Step 2: Coarse Search

3 Design and Implementation

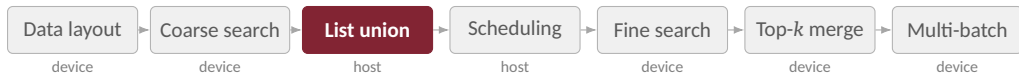


- Regular, dense: runs on the device
- Query tiles stay in L1, centroid blocks of 32 streamed from DRAM
- Running top-32 per query, so $N_{\text{probe}} \leq 32$
- Example: E replaces D for q_2 in block 2



Step 3: Batch-Level List Union

3 Design and Implementation



- The host collects the lists selected by all queries of a batch into one set

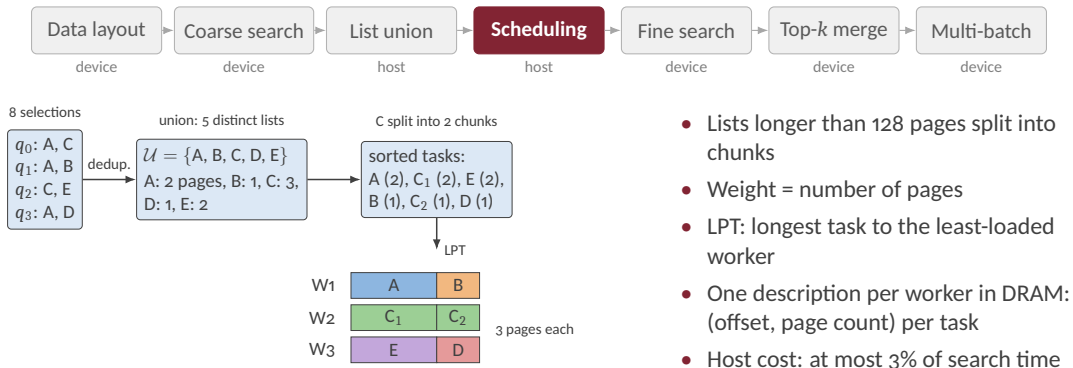
$$\mathcal{U}_b = \bigcup_r \mathcal{P}_{b,r} \quad \text{e.g. 8 selections} \rightarrow \{A, B, C, D, E\}$$

- Why: a page costs one tile product whether 1 or 32 queries need it
 - Consequence: each query is also scored against the lists of the other queries
- With queries in dataset order, batches share few lists: each query scans **12-32**× the vectors of its own lists



Step 4: Scheduling on 63 Workers

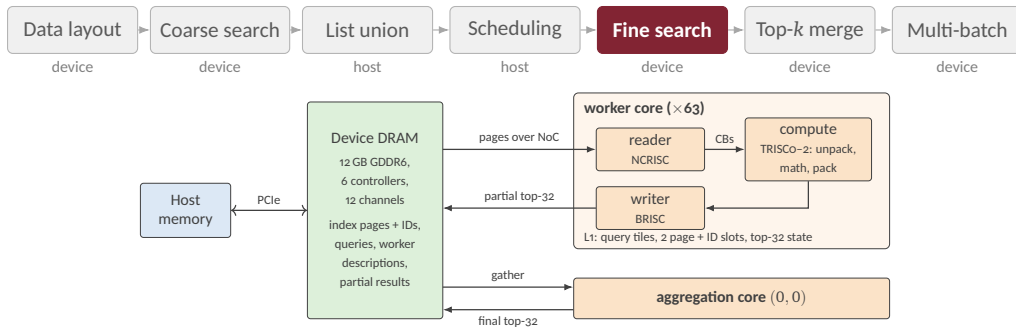
3 Design and Implementation





Step 5: Fine Search – Data Movement

3 Design and Implementation

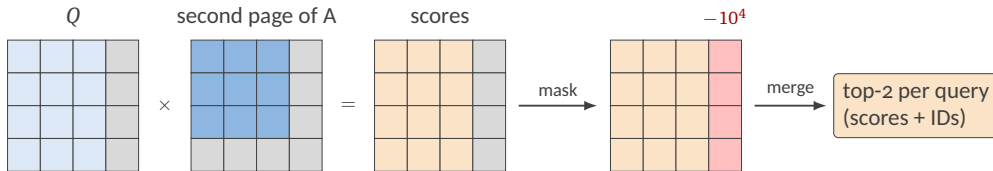
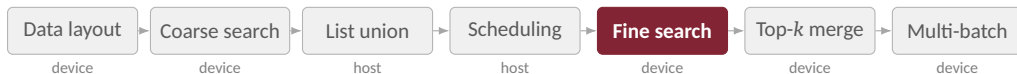


- Reader (NCRISC) streams pages over the NoC, double-buffered
- Compute (TRISCO-2) scores the page; writer (BRISC) stages results in DRAM



Step 5: Fine Search – One Page

3 Design and Implementation

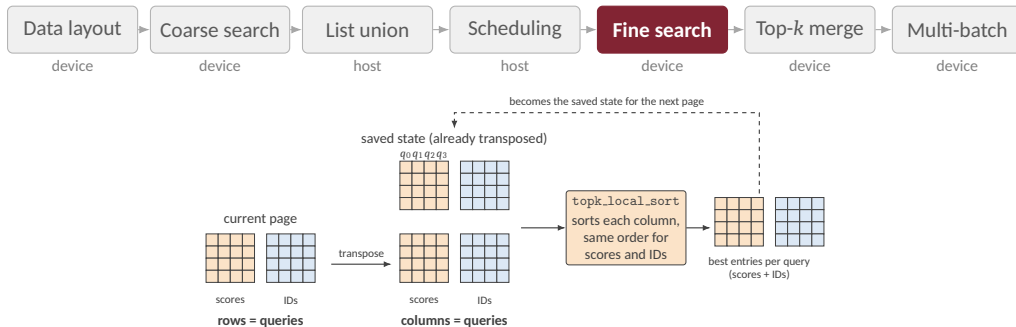


- Multiply the query tiles by the page: one 32×32 score tile
- Last page of a list: padded slots recognized by their sentinel ID and set to -10000
- Scores merged into the running top-32 of each query



Step 5: Fine Search – Top-32 Update

3 Design and Implementation

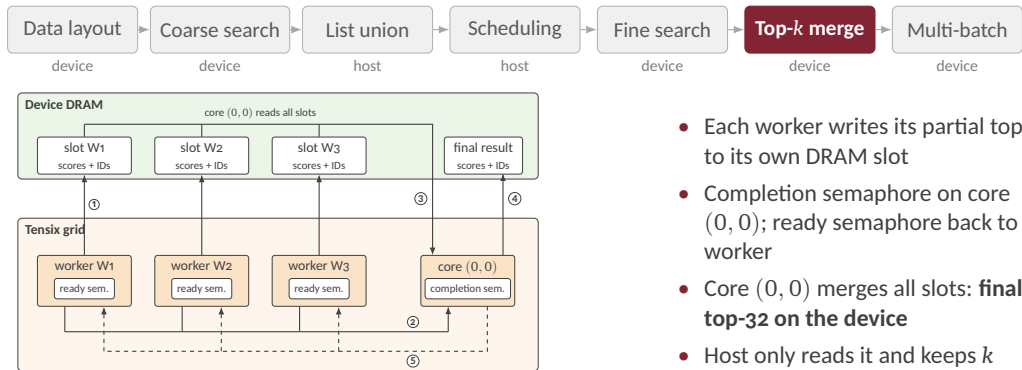


- Only the current page is transposed; the saved state is already kept transposed
- `topk_local_sort` orders each query column and moves scores and IDs together



Step 6: Hierarchical Top- k

3 Design and Implementation

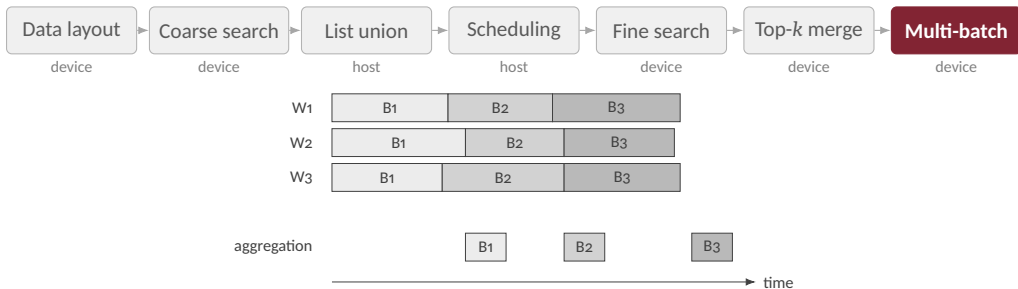


- Each worker writes its partial top-32 to its own DRAM slot
- Completion semaphore on core (0, 0); ready semaphore back to each worker
- Core (0, 0) merges all slots: **final top-32 on the device**
- Host only reads it and keeps k



Step 7: Multi-Batch Execution

3 Design and Implementation



- All batches of a workload run in one coarse and one fine invocation
- Workers continue with the next batch; aggregation follows behind
- Launch, synchronization, and host orchestration amortized over many batches



Table of Contents

4 Methodology

- ▶ Introduction
- ▶ Background
- ▶ Design and Implementation
- ▶ **Methodology**
- ▶ Results
- ▶ Conclusions



Experimental Setup

4 Methodology

- Dataset: glove-100-angular, 1,183,514 vectors, 100 dimensions, 10,000 queries
- **CPU:** Faiss IVF-SQ (FP16) on Leonardo, 56 and 112 cores
- **GPU:** Faiss IVF-SQ (FP16) on an NVIDIA A100
- **Tenstorrent:** IVF-Flat (FP16) on a Wormhole n150
- Same trained centroids for all systems



Matched Recall and Measurements

4 Methodology

- Grid of $(N_{\text{list}}, N_{\text{probe}})$ per system; compared at matched Recall@10: **60%, 78%, 88%, 95%**
- Throughput: median of 10 repetitions
- Energy: complete 100,000-query workload, **including initialization**, median of 10 sessions
 - host: RAPL package counters
 - accelerators: `nvidia-smi` (board) and `tt-smi` (chip) power, integrated over time



Table of Contents

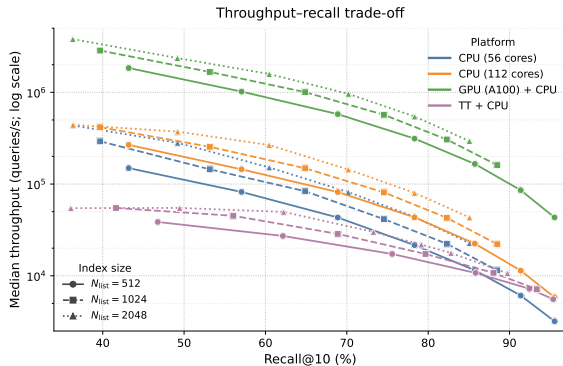
5 Results

- ▶ Introduction
- ▶ Background
- ▶ Design and Implementation
- ▶ Methodology
- ▶ **Results**
- ▶ Conclusions



Throughput vs. Recall

5 Results





Throughput at Matched Recall

5 Results

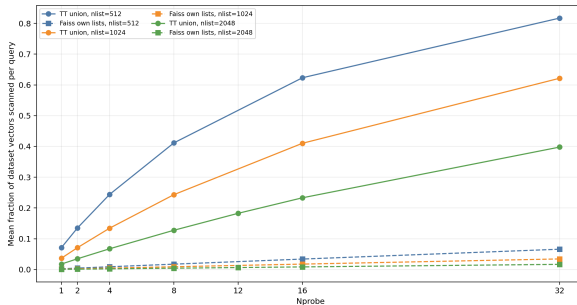
Target	Median QPS				Ratio to TT	
	CPU 56	CPU 112	A100	TT	CPU 56	A100
60%	151,858	266,184	1,583,712	49,720	3.05	31.85
78%	43,864	80,075	544,703	21,907	2.00	24.86
88%	11,466	22,079	160,799	10,762	1.07	14.94
95%	3,190	5,822	43,271	5,550	0.57	7.80

A100 always fastest; Tenstorrent declines more slowly and is **1.74**× the 56-core CPU at 95%



Effect of the Batch Union

5 Results

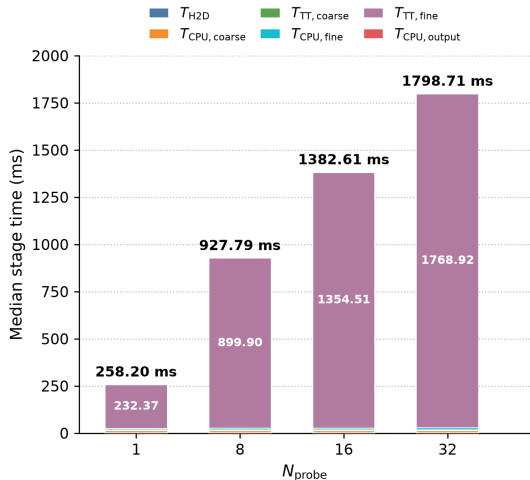


Each query scans $12-32 \times$ the vectors of its own lists



Where the Time Goes

5 Results



- $N_{\text{list}} = 512$, per 10,000 queries
- Fine search: **90–98%** of the search time
- Host steps and coarse search: under 30 ms
- Memory-bound: about 21 operations per byte vs. a balance point of 257



Device Timeline

5 Results

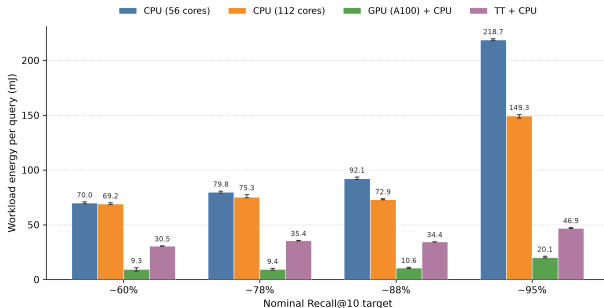


Worker imbalance sets batch completion; the aggregation core follows the slowest worker



Workload Energy

5 Results



- A100 system: lowest energy at every target
- Tenstorrent system: **53–79% less** than the CPUs, $2.3\text{--}3.8\times$ the A100
- The Wormhole chip is only **11–18%** of its system energy: the host dominates



Throughput vs. Energy

5 Results

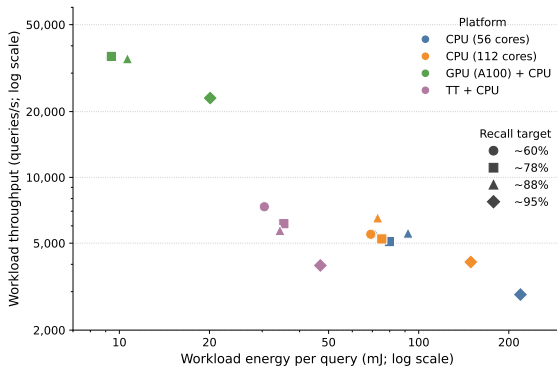




Table of Contents

6 Conclusions

- ▶ Introduction
- ▶ Background
- ▶ Design and Implementation
- ▶ Methodology
- ▶ Results
- ▶ **Conclusions**



Answers to the Research Questions

6 Conclusions

1. **Index:** IVF-Flat fits the tile model best
2. **Implementation:** feasible, 95.4% Recall@10 at 5,550 QPS; the batch union is the main inefficiency
3. **Performance:** A100 fastest (7.8–31.9 $\times$); Tenstorrent overtakes the 56-core CPU at high recall
4. **Energy:** far below the CPUs, above the A100, dominated by the host



Future Work

6 Conclusions

- **Group queries before batching:** first test at 512 lists, fine search -57% ($N_{\text{probe}} = 16$) and -42% ($N_{\text{probe}} = 32$)
- Lighter top- k update per page
- Better load balancing across batches and workers
- Index structures designed for the device



Thank you for listening!
Any questions?